

Práctica 2 SIW - Documentación

Pelayo Sierra Lobo UO294217

Web Crawler

El *script* implementado es un Web Crawler simple que descarga páginas web de manera recursiva y las ordena en archivos **WARC** agrupados por dominio. Cuenta con dos configuraciones, siendo estas búsqueda en profundidad (dfs) y búsqueda en anchura (bfs).

Para utilizar el crawler, además de tener instaladas todas las librerías necesarias (preferiblemente en un entorno virtual), hay que configurar el siguiente código:

```
file_name = 'crawler/seeds.txt'

algorithm = 'bfs' # dfs - bfs

robot_compliance = False

process_seeds(file_name, 10, 2, algorithm, robot_compliance)
```

- En *file_name*, introducimos la dirección de nuestras semillas. Deberían estar en la misma carpeta que **crawler.py**.
- En *algorithm*, elegimos entre **bfs** y **dfs** para cambiar el algoritmo.
- En *robot_compliance* establecemos la obediencia del crawler hacia el robots.txt de las páginas web
- Por último, se hace una llamada a *process_seeds*, añadiendo los dos parámetros previos y luego dos enteros representando el número máximo de archivos a descargar y los segundos entre repetición.

El método *process_seeds* lee el archivo con semillas y hace las llamadas a los algoritmos crawl dependiendo de la configuración y hasta que el número de páginas descargadas sea el establecido por parámetro.

Una vez se inicia el programa, se crea un directorio **pages** que contendrá los archivos **WARC** con las páginas web descargadas. Si se vuelve a ejecutar el programa, el propio script elimina esta carpeta y vuelve a crearla con los nuevos archivos que se van añadiendo.

La terminal informa a lo largo de la ejecución del estado del crawling, ofreciendo datos como la semilla que se está explorando en ese momento y las páginas que han sido descargadas con éxito. Al terminar, se genera un informe con los links de las páginas descargadas.

Función de procesamiento de semillas

`process_seeds` es la función de entrada principal del crawler. Se encarga de leer las semillas iniciales, limpiar resultados anteriores (eliminando la carpeta `pages/`) e inicializar el archivo `links.txt` con la información de ejecución y parámetros introducidos.

Dependiendo del archivo que haya sido seleccionado, invoca el crawler de búsqueda **en profundidad** o **en anchura**.

Al finalizar la ejecución, guarda un resumen con el número de páginas rastreadas y las URLs que fueron bloqueadas por `robots.txt` (en caso de tener `robot_compliance = True`).

Para comprobar si una URL puede ser rastreada según las reglas de `robots.txt`, se llama a la función `can_fetch(url)`, la cual descarga el archivo robots del sitio web y, utilizando el agente de usuario definido (en este caso "SIWbot"), retorna `True` si es posible y `False` en caso contrario.

Búsqueda en profundidad

Para la búsqueda en profundidad, se utiliza la función `crawldfs`:

- Parte de una URL y se adentra recursivamente en los enlaces que encuentra.
- Evita repetir las URLs ya exploradas mediante el set `explored`.
- Respeta el límite de profundidad (`DEPTH_LIMIT`) y el número máximo de páginas (`max_num`).
- Si `robot_compliance` está activado, sólo sigue los enlaces permitidos por `robots.txt`.
- Útil para explorar sitios "hacia dentro" de forma exhaustiva.

El límite de profundidad fue incluido para evitar que el algoritmo profundice en una dirección única indefinidamente, para que así tenga cierto grado de movimiento horizontal en el árbol de páginas. Este límite se puede encontrar en la constante `DEPTH_LIMIT` localizada al inicio del script, cuyo valor está preestablecido como 5.

Búsqueda en anchura

Para la búsqueda en anchura, se utiliza la función `crawl bfs`:

- Empieza con todas las semillas iniciales y las mete en una cola (`frontier`).
- Extrae URLs una por una y añade a la cola los enlaces descubiertos.
- De este modo, explora primero las páginas cercanas a las semillas antes de profundizar más.
- Igual que en DFS, respeta el límite de páginas y las reglas de `robots.txt` si están activadas.
- Además, también implementa el set `explored`.
- Es útil para obtener una visión más amplia de un sitio o varios sitios al mismo tiempo.

Descarga de archivos

Para la obtención de los archivos HTML, se utiliza la función `fetch_url` :

- Verifica si la URL contiene contenido HTML mediante `is_html` . Esta función comprueba que el `content-type` del *header* sea `text/html` . En caso negativo, muestra un mensaje de monitorización por pantalla.
- Si es válida, guarda la respuesta HTTP en un archivo **WARC comprimido** dentro de la carpeta `pages/` . Para esto, utiliza el método `capture_http` , donde también se informa de la profundidad en la que se haya el link encontrado.
- Registra el enlace en `links.txt`
- Devuelve el contenido procesado con BeautifulSoup para extraer enlaces.
- Añade un delay (`sleep`) configurable entre peticiones para no sobrecargar los servidores.

Resultados

A continuación se adjuntan los resultados obtenidos tras experimentar con las distintas configuraciones del crawler. Estos resultados son los reflejados en el archivo `links.txt` tras la ejecución. Al ejecutar el programa, los archivos **WARC** generados contendrán estos mismos archivos HTML bajo el código **200**, y se pueden visualizar utilizando la herramienta `replayweb.page` (<https://replayweb.page/>). Estos archivos pueden contener también links con códigos **301** y **302**, los cuales representan redirecciones.

Primer escenario

En este escenario se ejecutó el script sin obediencia hacia el `robots.txt`, desde las semillas `seeds.txt` . Tanto para este como para los otros escenarios, el límite de profundidad utilizado en DFS es 5.

BFS

```
Max number of pages to crawl: 10
Crawl delay between requests: 2 seconds
Crawling algorithm: BFS
Robots.txt compliance: disabled
```

Crawled Links:

```
https://www.uc3m.es/inicio
https://en.wikipedia.org/wiki/Dromaeosauroides
https://www.geeksforgeeks.org/python/python-string-strip/
https://www.uc3m.es/inicio/informacion-legal/politica-cookies
https://www.uc3m.es/home
https://sede.uc3m.es
http://intranet.uc3m.es
http://aulaglobal.uc3m.es
```

```
https://www.uc3m.es/conocenos  
https://www.uc3m.es/estudios
```

Total pages crawled: 10

Total URLs blocked by robots.txt: 0

DFS

Max number of pages to crawl: 10

Crawl delay between requests: 2 seconds

Crawling algorithm: DFS

Robots.txt compliance: disabled

Crawled Links:

```
https://www.uc3m.es/inicio  
https://www.uc3m.es/inicio/informacion-legal/politica-cookies  
https://www.uc3m.es/home/legal-information/cookies-policy  
https://www.uc3m.es/home  
https://sede.uc3m.es  
http://intranet.uc3m.es  
http://aulaglobal.uc3m.es  
https://www.uc3m.es/about-uc3m  
https://www.uc3m.es/studies  
https://www.uc3m.es/admission
```

Total pages crawled: 10

Total URLs blocked by robots.txt: 0

Podemos observar que para ambos algoritmos los resultados son diferentes. La diferencia más notable es que BFS explora todos los links de `seeds.txt`, mientras que DFS solo expande los enlaces relacionados con la primera semilla. Este es el principal problema de DFS en su aplicación en los crawlers web.

Segundo escenario

Se utiliza el archivo de semillas `seeds2.txt` con un límite de páginas 30 y sin obediencia de `robots.txt`

BFS

Max number of pages to crawl: 30

Crawl delay between requests: 2 seconds

Crawling algorithm: BFS

Robots.txt compliance: disabled

Crawled Links:

```
https://es.stardewvalleywiki.com/Stardew_Valley_Wiki  
https://www.realoviedo.es/noticias/aplazado-el-encuentro-entre-el-
```

valencia-cf-y-el-real-oviedo

<https://www.sanfer.es/>

https://es.stardewvalleywiki.com/Stardew_Valley_Wiki#mw-head

https://es.stardewvalleywiki.com/Stardew_Valley_Wiki#searchInput

https://es.stardewvalleywiki.com/Stardew_Valley

<https://es.stardewvalleywiki.com/Especial:Estad%C3%ADsticas>

<https://es.stardewvalleywiki.com/ConcernedApe>

<https://stardewvalley.net/>

<https://forums.stardewvalley.net/>

https://www.gog.com/game/stardew_valley

<https://www.humblebundle.com/store/stardew-valley>

https://store.steampowered.com/app/413150/Stardew_Valley/?l=spanish

<https://www.stardewvalley.net/stardew-valley-1-6-update-is-out-now-on-pc/>

<https://www.nintendo.com/games/detail/stardew-valley-switch>

<https://apps.apple.com/es/app/stardew-valley/id1406710800>

https://store.playstation.com/es-es/product/EP2319-CUSA26625_00-2010147637670638

<https://play.google.com/store/apps/details?id=com.chucklefish.stardewvalley&hl=es-ES>

<https://www.microsoft.com/es-es/p/stardew-valley/C3D891Z6TNQM>

https://es.stardewvalleywiki.com/Historial_de_versiones

https://es.stardewvalleywiki.com/Historial_de_versiones_de_consolas#PlayStation_4

<https://www.microsoft.com/es-es/store/p/stardew-valley/c3d891z6tnqm>

https://es.stardewvalleywiki.com/Historial_de_versiones_de_consolas#Xbox_One

https://es.stardewvalleywiki.com/Historial_de_versiones_de_consolas#Switch

https://es.stardewvalleywiki.com/Historial_de_versiones_de_consolas#PS_Vita

https://es.stardewvalleywiki.com/Historial_de_versiones_m%C3%B3viles#iOS

https://es.stardewvalleywiki.com/Historial_de_versiones_m%C3%B3viles#Android

https://de.stardewvalleywiki.com/Stardew_Valley_Wiki

https://stardewvalleywiki.com/Stardew_Valley_Wiki

https://fr.stardewvalleywiki.com/Stardew_Valley_Wiki

Total pages crawled: 30

Total URLs blocked by robots.txt: 0

DFS

Max number of pages to crawl: 30

Crawl delay between requests: 2 seconds

Crawling algorithm: DFS

Robots.txt compliance: disabled

Crawled Links:

```
https://es.stardewvalleywiki.com/Stardew_Valley_Wiki
https://es.stardewvalleywiki.com/Stardew_Valley_Wiki#mw-head
https://es.stardewvalleywiki.com/Stardew_Valley_Wiki#searchInput
https://es.stardewvalleywiki.com/Stardew_Valley
https://es.stardewvalleywiki.com/Stardew_Valley#mw-head
https://es.stardewvalleywiki.com/Stardew_Valley#searchInput
https://es.stardewvalleywiki.com/Archivo:Main_Logo.png
https://es.stardewvalleywiki.com/ConcernedApe
https://es.stardewvalleywiki.com/Stardew_Valley#cite_note-1
http://chucklefish.org
http://playstarbound.com
https://es.stardewvalleywiki.com/Stardew_Valley#cite_note-2
https://es.stardewvalleywiki.com/Stardew_Valley#cite_note-3
https://es.stardewvalleywiki.com/Stardew_Valley#cite_note-4
https://es.stardewvalleywiki.com/Stardew_Valley#cite_note-5
https://es.stardewvalleywiki.com/Stardew_Valley#cite_note-6
https://es.stardewvalleywiki.com/Multijugador
https://es.stardewvalleywiki.com/Stardew_Valley#cite_note-7
https://stardewvalley.net/
http://blog.chucklefish.org/shop/#Stardew
http://upload.farm/
http://community.playstarbound.com/forums/stardew-valley.72/
http://store.steampowered.com/app/413150
http://steamcommunity.com/app/413150
https://es.stardewvalleywiki.com/Stardew_Valley#cite_ref-1
http://cornellsun.com/2016/02/23/stardew-valley-pushing-the-boundaries-
of-farming-rpgs/
https://es.stardewvalleywiki.com/Stardew_Valley#cite_ref-2
https://es.stardewvalleywiki.com/Stardew_Valley#cite_ref-3
http://www.ubergizmo.com/2016/07/stardew-valley-available-mac-linux/
https://es.stardewvalleywiki.com/Stardew_Valley#cite_ref-4
```

Total pages crawled: 30

Total URLs blocked by robots.txt: 0

De nuevo se puede observar la diferencia entre ambos algoritmos. Al no obedecer a robots, el tiempo de ejecución es aceptable tal y como está implementado.

Tercer escenario

Para este escenario se añade la obediencia hacia los archivos *robots.txt*. Utilizaremos las semillas *seeds3.txt*, que contienen dos links. Además, por motivos de tiempo en este caso descargaremos 8 archivos totales. Esto se debe a que muchos enlaces producen bloqueos a través del archivo robots, por lo que tarda un tiempo excesivo sin implementar concurrencia (especialmente en el caso de **bfs**).

BFS

```
Max number of pages to crawl: 8
Crawl delay between requests: 2 seconds
Crawling algorithm: BFS
Robots.txt compliance: enabled
```

Crawled Links:

```
https://quotes.toscrape.com/
https://en.wikipedia.org/wiki/Fanta_cake
https://quotes.toscrape.com/login
https://quotes.toscrape.com/author/Albert-Einstein
https://quotes.toscrape.com/tag/change/page/1/
https://quotes.toscrape.com/tag/deep-thoughts/page/1/
https://quotes.toscrape.com/tag/thinking/page/1/
https://quotes.toscrape.com/tag/world/page/1/
```

```
Total pages crawled: 8
Total URLs blocked by robots.txt: 407
```

Podemos observar que se han bloqueado un total de 407 URLs, muchas probablemente pertenecientes a Wikipedia.

En la siguiente ejecución, cambié de posición los links del archivo (el primero por el segundo), y reduje el número a 6. El motivo es que, de lo contrario, el algoritmo DFS no terminaba nunca a la par que se veía bloqueado constantemente por el robots.txt de alguna página web.

DFS

```
Max number of pages to crawl: 6
Crawl delay between requests: 2 seconds
Crawling algorithm: DFS
Robots.txt compliance: enabled
```

Crawled Links:

```
https://en.wikipedia.org/wiki/Fanta_cake
https://www.oetker.de/recipes/r/fanta-kuchen-mit-schmand
https://www.oetker.de/de_DE/index
https://www.oetker.de/hilfe/kontakt
https://www.oetker.de/
https://cloud.email.oetker.com/de-de-contact
```

```
Total pages crawled: 6
Total URLs blocked by robots.txt: 131
```

Este último escenario sirve para demostrar la dificultad de implementar *crawlers* éticos que sigan las especificaciones de las páginas. Al evitar el archivo *robots.txt*, podemos obtener muchos más archivos en mucho menos tiempo, pero el uso de robots evita sobrecargar los

servidores de las páginas web y por esto se considera una buena práctica, a pesar de que suponga tiempos excesivos en la ejecución (por lo menos en casos como este en el que métodos de optimización como concurrencia no han sido utilizados).